

hw6: implementing signals

Due date: Mon Nov 13, at noon.

In this homework, we add support for standard Unix signals to xv6. As in past homeworks, start with the "signalhw" template (signalhw instead of hw6 to avoid confusion after a renumbering) from the class repository (cs461). Refer to lectures, the text book and Linux man pages, including: [man sigreturn](#), as well as [this linuxjournal article](#) to learn more about signals.

alarm() - send SIGALRM to a process after a set interval

The template contains a default handler for all signals: kill the process. In this part, we implement a new system call (`alarm(seconds)`), which sends SIGALRM to the calling process after the specified time interval.

The program alarmtest.c calls alarm(), then enters an infinite loop. A correct implementation of alarm() kills the process after the specified number of seconds. You'll want store the alarm time in the process, then check if any process needs an alarm signal, every time the timer goes off.

signal() step 1 - change the disposition of a signal

The signal() system call can be used to change the “disposition” of a signal, i.e. how the signal is handled when received. In this step, you need to support only SIG_DFL (=0, default), and SIG_IGN (=1, ignore). The template already has built-in handling for SIG_DFL (kill the process), so you need only support the configuration, and “handle” signals with a SIG_IGN disposition.

You'll want to store the "disposition" of each signal in proc.h, so that check_signals can act accordingly when a signal has arrived.

Test your implementation using alarmtest2.c.

signal() step 2 - user space signal handling functions

If the second argument is a function (i.e. a value that is not SIG_DFL or SIG_IGN), then the function passed in should be called when the signal is received. We will follow the Linux design for implementing this, which is described in the linuxjournal article above, and discussed in lecture.

A skeleton version of sys_sigret is provided. For it to work right, you'll need to store and restore the trap frame. Add field to keep a backup copy of the trapframe in struct proc, so that trapret can restore the registers before returning to userspace.

VERY IMPORTANT: the signal handler must run in user mode only

Test your implementation using alarmtest3.c.

Ctrl-C sends SIGINT to the foreground process

Implement the system call fgproc(), and modify sh.c to use it to track the foreground process. Whenever CTRL-C is pressed, send SIGINT to the foreground process.

Test your implementation with helloloop.c or any other program that continues running.

Some hints

There are two main things that are challenging in this homework. One isn't a big deal if you're familiar with casting and function pointers, but I suspect dealing with signal handler addresses (and numbers, for the default and ignore actions) is going to pose a basic syntactical challenge for some.

The other part is setting up the stack for the signal handler function. You'll want it to look like this,

- other stack frames
- a few bytes of machine code to call sys_sigret
- address of the first byte of the code above

You'll want %rsp to be pointing at the end of that address by the time you return to userspace (and %rip to the handler function address). The system call return code (syscall_trapret in trapasm.S) takes %rsp from the trap frame. Strangely enough, the sysret instruction takes %rip from %rcx, so you'll want to put the instruction pointer in the %rcx field in the trapframe.

As a hint for the machine code bit, the user space symbol “sigret” points at the exact code you need.

turn-in

Use this [invitation link](#) to create your turn-in repository. Push your turn-in branch as hw6.